FINAL CAPSTONE PROJECT • SESSION 21

TaskBoard: Cloud-Native DevSecOps Platform

Automated Multi-Stage CI/CD, Container Security Gates, AWS Infrastructure as Code, Kubernetes Helm Orchestration & SRE Observability

🚀 Application

- FastAPI + React 18 + PostgreSQL
- Alembic database migrations
- Non-root user container isolation

🛡️ DevSecOps

- GitHub Actions CI/CD
- Pytest automated tests
- Trivy scan for HIGH/CRIT CVEs

☸️ Kubernetes & Helm

- Dual-replica deployments
- Ingress routing + HPA auto-scale
- Liveness & Readiness probes

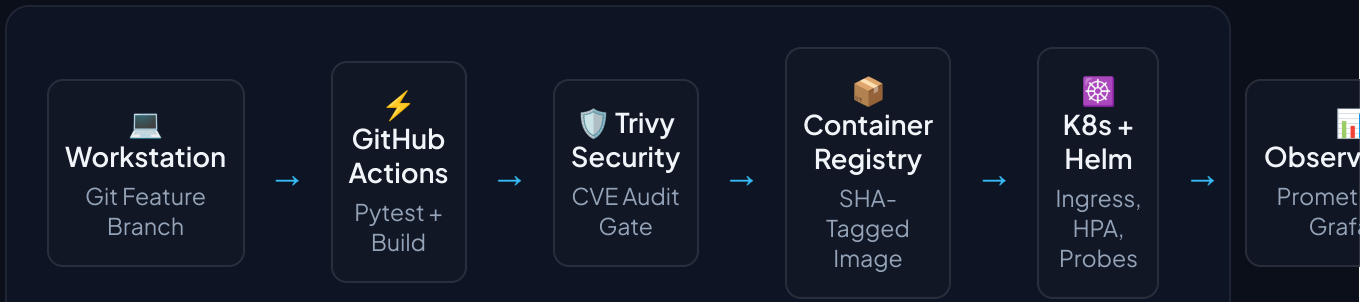
📊 Cloud & SRE

- Modular AWS VPC & EKS IaC
- Prometheus /metrics scraping
- Grafana latency & throughput

SYSTEM ARCHITECTURE

End-to-End DevSecOps System Architecture

A seamless journey from code commit to hardened, auto-scaling Kubernetes cluster



Delivery & Security Plane

- **Automated Testing:** `pytest` validates REST endpoints & database integrity before build.
- **Vulnerability Filtering:** Trivy scans containers for OS and package CVEs, halting on criticals.
- **Immutable Versioning:** Images tagged with `${{ github.sha }}` guaranteeing traceable rollbacks.
- **Ephemeral Smoke Test:** CI provisions temporary `Kind` cluster to verify pod health.

Cloud Runtime & Observability Plane

- **Infrastructure as Code:** Terraform provisions AWS VPC, multi-AZ subnets, and EKS 1.31 cluster.
- **Declarative Orchestration:** Custom Helm chart manages deployments, ConfigMaps, and Secrets.
- **Traffic Routing:** Nginx Ingress routes `/` to React and `/api` to FastAPI.
- **Telemetry:** Prometheus scrapes live metrics while Grafana visualizes RED metrics.

APPLICATION & CONTAINERIZATION

Application Architecture & Container Hardening

Multi-tier microservice designed for least-privilege security and native health probing

FastAPI Backend (Python)

- Asynchronous Python 3.12 with Uvicorn server
- 6 Core REST endpoints: GET, POST, PUT, DELETE, and `/api/tasks/stats`
- SQLAlchemy 2.0 ORM with PostgreSQL database
- Alembic versioned database migrations
- Prometheus middleware for runtime metrics

React UI & Routing

- Interactive Kanban board built with React 18 & Vite
- Real-time task breakdown: Total, To-Do, In-Progress, Done
- Status filters and modal task creator
- Nginx reverse proxy handles transparent `/api` routing
- Loading spinners and graceful error banners

Container Hardening

- **Non-Root User:** Runs strictly as `appuser (UID 10001)`
- **Multi-Stage Builds:** Strips build tools, compilers, and dev dependencies from final image
- **Minimal Footprint:** Built on Alpine base images
- **Local Compose:** `docker compose up --build` spins up full stack with 1 command

AUTOMATION & DEVSECOPS

Automated CI/CD & DevSecOps Security Gates

Shift-left quality and container vulnerability gates in GitHub Actions (.github/workflows/ci-cd.yml)

Quality Gate: Automated Pytest Suite

- Executes on every Git push prior to container image creation
- Validates task creation, status advancement, and statistics calculations
- Uses isolated SQLite mock database to prevent test pollution
- **Fail-Fast Behavior:** Broken tests halt pipeline immediately before resource consumption

Security Gate: Trivy Vulnerability Scan

- Performs automated CVE scans on backend and frontend images
- Configured to flag and break pipeline on `HIGH` or `CRITICAL` findings
- Scans OS packages and application dependencies
- Written CVE analysis documents all evaluated vulnerabilities

Artifact Publishing: Immutable SHA Tags

- Images tagged with exact Git commit hash `${{ github.sha }}`
- Eliminates ambiguity caused by the mutable `latest` tag
- Pushed directly to GitHub Container Registry (GHCR)

Verification: Ephemeral Kind Cluster

- Spins up a throwaway Kubernetes cluster within GitHub Actions runner
- Loads newly-built SHA images and applies manifests
- Awaits condition `kubectl wait --for=condition=ready`

INFRASTRUCTURE AS CODE

Terraform: AWS Cloud Infrastructure Automation

Declarative provisioning of AWS VPC and Managed EKS Kubernetes Cluster

Resilient AWS VPC Design

- Multi-AZ networking in region `ap-south-1`
- Dual public and private subnets across AZs
- Internet Gateway for public ingress routing
- NAT Gateway enabling outbound container updates
- Strict Security Groups adhering

Managed AWS EKS 1.31 Cluster

- Production-ready managed Kubernetes control plane
- Node Group: 2× `t3.medium` EC2 worker instances
- AWS IAM roles and OIDC authentication provider
- EBS CSI Driver storage provisioner for database volumes
- Native cluster autoscaler integration

Secure & Reproducible Lifecycle

- Declarative HCL: `versions.tf`, `variables.tf`, `outputs.tf`
- Includes `terraform.tfvars.example` template
- Zero hardcoded credentials or cloud secrets in Git
- Validated `terraform destroy` lifecycle prevents runaway cloud expenses

KUBERNETES & ORCHESTRATION

Kubernetes Workloads & Helm Packaging

Production-ready Helm packaging, Ingress networking, and autonomous autoscaling

Helm Packaging (helm/taskboard)

- Clean chart structure: `Chart.yaml`, `values.yaml`, and templates
- Environment parameterization (development vs. production values)
- Single-command deployment: `helm upgrade --install taskboard ./helm/taskboard`
- Decoupled ConfigMaps (`taskboard-config`) and Secrets (`taskboard-db`)

Health Probes & Zero-Downtime Rollouts

- **Liveness Probe:** Checks `/health`; restarts container if deadlocked
- **Readiness Probe:** Verifies DB connectivity at `/ready` before routing requests
- **Rolling Update Strategy:** `maxUnavailable: 0` and `maxSurge: 1` guarantees zero downtime
- Dual-replica baseline guarantees high availability

Ingress Routing & Service Discovery

- Ingress resource routes traffic via hostname `taskboard.local`
- Path routing: `/` routes to Frontend; `/api` routes to Backend
- ClusterIP internal networking protects backend and

Horizontal Pod Autoscaler (HPA)

- Autoscaler dynamically monitors CPU utilization via Metrics Server
- Target CPU threshold: **60%**
- Autoscaling boundaries: **2 Min Replicas to 6 Max Replicas**

INCIDENT RESPONSE & TROUBLESHOOTING

Production Troubleshooting Challenge

Simulated root cause analysis and resolution workflows on broken Kubernetes resources

Challenge 1: ImagePullBackOff Incident

- **Symptom:** Pod stuck in `ErrImagePull` / `ImagePullBackOff` state.
- **Root Cause Analysis:** Executed `kubectl describe pod`. Identified that the deployment referenced an invalid, non-existent registry tag: `example.invalid/taskboard:nonexistent-tag`.
- **Remediation:** Patched container image directly using: `kubectl set image deployment/broken-image app=nginx:1.25-alpine`
- **Verification:** Pod successfully pulled image and transitioned to `Running (1/1)`.

Challenge 2: Service Selector Misconfiguration

- **Symptom:** Ingress returning HTTP 503; `kubectl get endpoints broken-service` returned `<none>`.
- **Root Cause Analysis:** Compared Service selector labels (`app: misconfigured-label`) against target Pod labels (`app: broken-image`). Discovered label mismatch.
- **Remediation:** Patched the Service selector to match active pod labels using: `kubectl patch service broken-service -p '{"spec":`

OBSERVABILITY & SRE

Observability: Telemetry, Metrics & SRE Dashboards

Continuous visibility into cluster health and application performance using Prometheus and Grafana

Prometheus Instrumentation

- FastAPI client automatically instruments all HTTP traffic
- Exposes standardized Prometheus metrics at `/metrics`
- Prometheus server scrapes targets via Kubernetes ServiceMonitors
- Health status verified with targets showing `UP` in Prometheus console

Grafana SRE Dashboards

- **Throughput:** Total HTTP requests per second partitioned by endpoint
- **Latency:** p95 and p99 request duration histograms
- **Error Budget:** Ratio of 2xx success codes vs 4xx/5xx failures
- **Infrastructure:** CPU and memory utilization per pod

Load Testing & Dynamic Scaling

- Executed synthetic load script: `./scripts/load-test.sh`
- Observed real-time request spikes on Grafana panels
- Monitored HPA scaling up backend replicas as CPU crossed 60%
- Confirmed zero dropped packets or downtime during scaling cycle

EVALUATION & CONCLUSION

Evaluation Summary & Key Engineering Takeaways

Comprehensive 100/100 point rubric alignment and engineering principles mastered

100-Point Grading Rubric Alignment

- **M1 Application (10 pts):** FastAPI + React + PostgreSQL + Alembic
- **M2 Testing (10 pts):** Pytest suite covering all CRUD endpoints
- **M3–M4 Git & Docker (15 pts):** Multi-stage non-root containers
- **M5–M6 CI/CD & DevSecOps (20 pts):** GitHub Actions + Trivy scans
- **M7 Terraform IaC (15 pts):** AWS VPC + Managed EKS cluster
- **M8 Kubernetes & Helm (15 pts):** Ingress + Dual Replicas + HPA
- **M9 Observability (10 pts):** Prometheus scraping + Grafana SRE metrics
- **M10 Presentation (5 pts):** Complete documentation and video walkthrough

Core Principles Demonstrated

- **Shift-Left Security:** Automated Trivy scans prevent vulnerable images from ever reaching the registry.
- **Immutable Deliverables:** Tagging images with commit SHAs eliminates drift and enables instantaneous rollbacks.
- **Infrastructure as Code:** Zero manual console steps; reproducible cloud provisioning via Terraform.
- **Resilient Operations:** Probes, HPA, and self-healing deployments maintain high availability under load.